

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/338285571>

A Flexible and Efficient Real-Time ORB-Based Full-HD Image Feature Extraction Accelerator

Article in IEEE Transactions on Very Large Scale Integration (VLSI) Systems · December 2019

DOI: 10.1109/TVLSI.2019.2945982

CITATIONS

23

READS

188

7 authors, including:



Rongdi Sun

Shanghai Jiao Tong University

14 PUBLICATIONS 67 CITATIONS

[SEE PROFILE](#)



Jiuchao Qian

Shanghai Jiao Tong University

44 PUBLICATIONS 531 CITATIONS

[SEE PROFILE](#)



José Leonardo Romero Hung

Shanghai Jiao Tong University

4 PUBLICATIONS 32 CITATIONS

[SEE PROFILE](#)



Zheng Gong

China Academy of Information and Communications Technology

22 PUBLICATIONS 175 CITATIONS

[SEE PROFILE](#)

Some of the authors of this publication are also working on these related projects:



levle crossing ADC [View project](#)

A Flexible and Efficient Real-Time ORB-Based Full-HD Image Feature Extraction Accelerator

Rongdi Sun[✉], Jiuchao Qian[✉], Romero Hung Jose, Zheng Gong[✉], Ruihang Miao[✉], Wuyang Xue[✉],
and Peilin Liu[✉], *Member, IEEE*

Abstract—This article proposes a hardware accelerator for image feature extraction based on the oriented features from accelerated segment test (FAST) and rotated binary robust independent elementary features (ORB) algorithm. The architecture adopts a hybrid workflow to process three scales in parallel as an octave, and multiple octaves are processed in serial by time-sharing the DRAM. In order to support any image resolution, a block-wise dataflow is used on the serial-parallel architecture. The overlapped data between two blocks is reused. As a result, the on-chip memory is limited to 1.47 Mb, and the DRAM bandwidth is compressed by 33%. Besides, 3×3 non-maximum suppression with heap sorting is applied to balance keypoint distribution in 2-D. It improves the valid keypoint match ratio by 7.09%. In addition, parallel processing in keypoint detection increases 2× throughput. Moreover, approximate computing and superscalar processing reduce the timing cost in orientation estimation and descriptor generation by 92.78% and 67% respectively. Compared to the non-optimized baseline architecture, the proposed architecture saves the total timing cost by 82.4%. The accelerator is implemented in a Xilinx MPSoC, and it achieves 108 fps on full-HD images at 200 MHz under 873 mW.

Index Terms—Feature extraction, hardware accelerator, oriented features from accelerated segment test (FAST) and rotated binary robust independent elementary features (ORB).

I. INTRODUCTION

IMAGE feature extraction is a fundamental task in the field of computer vision. Recently, high-level features learned from deep neural networks (DNNs) have become a great success in object detection and recognition. Still, in a variety of applications such as visual odometry (VO) and simultaneous localization and mapping (SLAM), handcrafted features are widely used due to their good matching accuracy at much less computation and memory cost than DNN features. Scale-invariant feature transform (SIFT) [1] is one of the most prominent blob-like features for 2-D grayscale images, which is invariant to scale, rotation, noise, and illumination. Likewise, speeded-up robust feature (SURF) [2] is proposed with lower complexity using the integral image, and it retains the most performance as SIFT. Both of them detect robust keypoints in multiple scales and generate descriptors for keypoints based on the histogram of gradients (HoG).

Manuscript received June 11, 2019; revised August 25, 2019 and September 18, 2019; accepted October 3, 2019. Date of publication December 31, 2019; date of current version January 21, 2020. This work was supported in part by the National Natural Science Foundation of China under Grant 61903246. (Corresponding author: Jiuchao Qian.)

The authors are with the School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: jcqian@sjtu.edu.cn).

Color versions of one or more of the figures in this article are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TVLSI.2019.2945982

Despite high-quality features obtained by SIFT or SURF, it is infeasible to process high-resolution images in real time. In order to address the issue, various binary descriptors have been designed as alternatives to SIFT and SURF. Typically, a binary descriptor is encoded as a high dimensional vector. Each bit of the vector is calculated by comparing the intensity of the sampled pixels. The binary descriptors has a simple computation pattern and requires a small amount of memory. A combination of features from accelerated segment test (FAST) [3] and binary robust independent elementary features (BRIEFs) [4] is widely used for keypoint detection and feature description, respectively, since it has comparable performance with SIFT. This method is further improved as oriented FAST and rotated BRIEF (ORB) [5] to achieve scale and viewpoint invariance. The ORB algorithm has been applied in a sparse visual SLAM system [6] successfully.

As the demand for embedded vision applications with limited power and computing resources grows, many researchers have implemented customized architecture to accelerate binary feature extraction for high-resolution images in real time. In [7], a bit-level parallel image recognition accelerator is proposed. It adopts a string searching method for corner detection, sharing a unified data path for both FAST and BRIEF units with no support for scale and rotation invariance. Lam *et al.* present an area-time efficient architecture for FAST and BRIEF detectors in [8]. It employs an optimized adder tree for the smoothing operation and a reusable sampling scheme for the descriptor generation. Zhu *et al.* [9] implement a hardware-friendly accelerator through joint architecture-algorithm design. A directional pairwise edge detector and an rotation invariance sample pattern are used to refine keypoints and efficiently compute descriptors. To reduce the computation complexity of the rotated BRIEF (rBRIEF) descriptor, Pham *et al.* [10] present an area-optimized architecture that combines a moving summation strategy with approximate computing in orientation estimation. Rather than implemented in ASIC, Fang *et al.* [11] accelerate ORB feature extraction on FPGA and study the effect of word length in orientation estimation. This article in [12] is another ORB feature extraction accelerator based on FPGA, in which the rBRIEF unit is duplicated for high resolution to avoid dropping keypoints. Ulusel *et al.* [13] present a comparative analysis of FAST detection along with rBRIEF description of various embedded platforms. It demonstrates that the FPGA implementation outperforms CPU and GPU in terms of both power and performance.

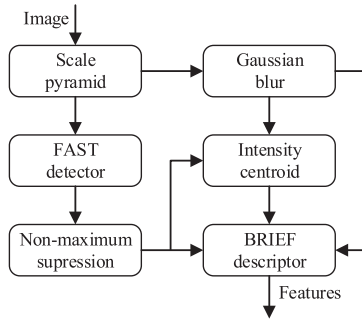


Fig. 1. ORB algorithm flow.

The above image feature extraction accelerators are either streaming architecture [8], [11]–[13] or nonstreaming architecture [7], [9]. The streaming architecture directly stores the source image to the on-chip line buffers row-wise. It usually has high throughput because of no external memory access for the intermediate data. Since the depth of the line buffers equals the image width, it is incompatible with a larger width. On the contrary, the nonstreaming architecture stores the source image to the DRAM and then loaded to the on-chip buffers. The nonstreaming architecture has a frame of latency due to DRAM access. By loading the data block-wise, it is easy to support any resolution. In this article, the nonstreaming architecture is taken as the baseline for three reasons. First, complex vision applications always run on high-performance SoCs, where DRAM is necessary to store frames and massive temporary data. Second, the source image is likely used not only for feature extraction, but also for other tasks in a multithread application. Third, we prefer a flexible architecture to support any resolution and any number of scales.

In this article, we present a real-time ORB feature extraction accelerator for visual SLAM applications. This article was initially introduced in [14] and [15] and now refined more like a coprocessor. Both works in [14] and [15] adopt streaming architecture for keypoint detection on three scales in parallel, where source images are stored in the on-chip line buffers, and nonstreaming architecture for descriptor generation, where smoothed images are offloaded to the DRAM for later use. In [14], the keypoint orientation is measured by comparing the absolute differences between the keypoint and the pixels on the Bresenham circle. In [15], a score recorder is added after the nonmax suppression (NMS) to filter poor-quality keypoints in the horizontal direction. Besides, smoothed images on every other scale are stored to reduce the DRAM access. Also, the orientation is calculated using the intensity centroid (IC) by a CORDIC unit. In this article, we take advantages of nonstreaming architecture and propose several optimizations in architecture, dataflow, memory, and computation to achieve flexibility and efficiency.

- 1) *Serial-Parallel Hybrid Workflow*: In the ORB algorithm, a source image is down-sampled iteratively to generate several scales. The architecture can process three scales in parallel as an octave, and more octaves in series by time sharing the DRAM, which is flexible to support any number of scales.

- 2) *Block-Wise Reusable Dataflow*: The architecture supports any resolution by loading the source image and the lowest scale of an octave from the DRAM block-wise. Besides, the overlapped data between two blocks are reused to save the on-chip memory and the DRAM bandwidth.
- 3) *Keypoint Distribution Balance*: In order to benefit the feature correspondence in visual SLAM applications, a 3×3 NMS unit with heap-sorting is used to balance the keypoint distribution in 2-D over an image.
- 4) *Bottleneck Computation Optimization*: The orientation estimation includes massive complex operations, and the descriptor generation has hundreds of random memory access. Hence, approximate computing, superscalar processing, and odd-even data interleaving are adopted to solve such bottlenecks.

The remainder of this article is organized as follows. In Section II, we briefly review the algorithm for image feature extraction and point out its challenges when deployed on embedded platforms. In Section III, we introduce the overall system architecture and its dataflow. Section IV describes the implementation and optimization of the critical modules. The experiment results are presented in Section V. Section VI concludes this article.

II. ALGORITHM REVIEW

A classic image feature extraction flow includes four main steps: 1) pyramid building; 2) keypoint detection; 3) orientation estimation; and 4) descriptor generation. The implementation details vary in accuracy and speed. We take the ORB algorithm shown in Fig. 1 as a guidance of the architecture design because it is simple and effective in visual SLAM applications [6].

The ORB algorithm first builds with the source image a scale pyramid, in which the higher scale is down-sampled from the previous lower scale. After this, the FAST corner detection locates the keypoint at each scale. Then the NMS filters those low-quality keypoints. Next, the orientation or the angle of each keypoint is computed using the IC method. Finally, each keypoint is encoded using the BRIEF descriptor rotated by the angle. Each step of a hardware-friendly ORB algorithm is described as follows.

A. Pyramid Building

The image pyramid can improve scale invariance of features because the same features can be extracted from similar scenes but captured at different distances. We use bilinear interpolation to perform down-sampling by a scaling factor of 1.25,¹ which approximates to the default value 1.2 in the OpenCV library [16]. As shown in Fig. 2(a), along the horizontal direction, every five pixels are down-sampled to four pixels. Specifically, the first one equals to its initial value, and the rest are taken from the weighted summation of two

¹In this article, the scaling factor denotes the downscaling ratio in one dimension. Hence, an image is actually down-sampled by the square of the scaling factor.

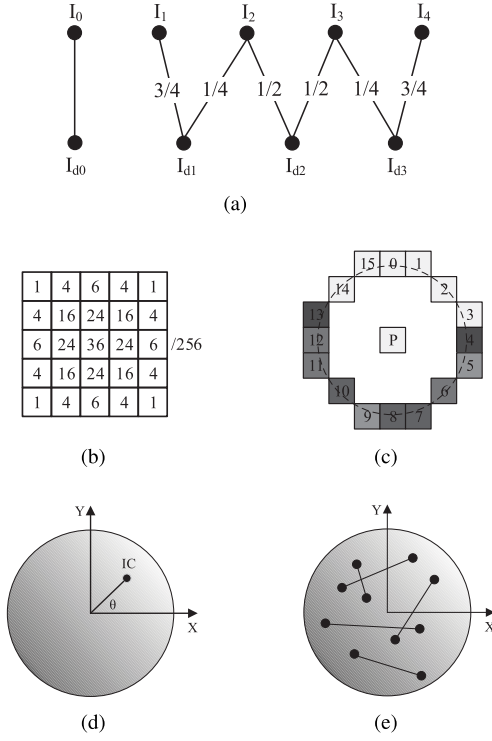


Fig. 2. Computation patterns of the hardware-oriented feature extraction. (a) Bilinear interpolation down-sampling. (b) Binomial filter. (c) FAST detector. (d) IC. (e) BRIEF descriptor.

neighbors. This process can be completed by addition and shift operations

$$\begin{cases} I_{d0} = I_0 \\ I_{d1} = ((I_1 \ll 1) + I_1 + I_2) \gg 2 \\ I_{d2} = (I_2 + I_3) \gg 1 \\ I_{d3} = (I_3 + (I_4 \ll 1) + I_4) \gg 2 \end{cases} \quad (1)$$

where I_0 to I_4 and I_{d0} to I_{d3} are the intensity of source pixels and down-sampled pixels, respectively. The results of horizontal down-sampling are then taken as inputs to vertical down-sampling to compute the final down-sampled image. Instead of Gaussian filtering in the original algorithm, the scale pyramid is smoothed by a 5×5 binomial coefficient filter for orientation estimation and descriptor generation [see Fig. 2(b)]. The binomial smoothing is also easily realized by addition and shift due to its numerical feature.

B. Keypoint Detection

Keypoint is a pixel with distinctive structure or color information in a local region. The FAST detector operates on a 16-pixel Bresenham circle centered at the candidate pixel P with an intensity of I_p , as shown in Fig. 2(c). If n contiguous pixels on the circle are brighter or darker than I_p by a threshold of t , then the center pixel P will be labeled as a corner. In [3], the author builds an ID3 decision tree as a classifier. Still, we perform FAST detection based on its definition because comparison operation is easy to implement in hardware. The optimal value of n is set to 9, which produces the best repeatability [3]. Meanwhile, the score of each corner

is calculated. It is defined by the sum of absolute intensity difference between the center pixel and those on the circle. The corner with the highest score compared to its 8-connected neighbors will pass the 3×3 NMS as a keypoint.

C. Orientation Estimation

To achieve viewpoint invariance, each keypoint is given an orientation. In the ORB algorithm, the orientation is estimated by IC in a local patch [see Fig. 2(d)]. The horizontal component m_x and vertical component m_y can be computed by sum of the intensity $I(x, y)$ weighted by the coordinate (x, y) :

$$\begin{cases} m_x = \sum_{x=-r}^r x I(x, y) \\ m_y = \sum_{y=-r}^r y I(x, y) \end{cases} \quad (2)$$

where the origin locates at the keypoint and the radius of the patch r is set to 15 by default. The orientation or the angle of the keypoint θ is defined by

$$\theta = \arctan\left(\frac{m_y}{m_x}\right) \quad (3)$$

where arctan is quadrant-aware. Apparently, hundreds of pixels and complex operations such as division and inverse trigonometric functions are required for the computation, which need to be solved by the accelerator.

D. Descriptor Generation

The descriptor is encoded to a long vector. The similarity of the two features can be measured by Hamming distance of the descriptors. The BRIEF descriptor is a 256-bit vector [4], where each bit is the comparison result $\tau(p; x, y)$ between the intensity of two pixels $p(x)$ $p(y)$ sampled from an area centered at the keypoint [see Fig. 2(e)]:

$$\tau(p; x, y) = \begin{cases} 1, & p(x) < p(y) \\ 0, & p(x) \geq p(y). \end{cases} \quad (4)$$

The ORB algorithm uses a learning method to select those pixel pairs with high variance and low correlation to achieve better performance. Then the coordinates of the sampled pixel pairs are rotated by the angle of the keypoint before the binary test. Even though the operations are relatively simple, this step is challenging for hardware implementation due to a large amount of random memory access. The accelerator exploits superscalar technique to reduce the processing time at a moderate cost.

III. SYSTEM ARCHITECTURE

As shown in Fig. 3(a), the overall system is composed of four major components: a feature extraction accelerator, a CPU, a DMA, and an external DRAM. The accelerator, CPU, and DMA are fabricated inside a Xilinx MPSoC chip and connected through AXI interfaces. A C-based embedded program runs on the host CPU to configure and control the other functional units. Because the accelerator is designed as a ready-to-use coprocessor, the commands running on the CPU is rather simple. The DMA transfers data between the DRAM and the accelerator, which extract ORB features.

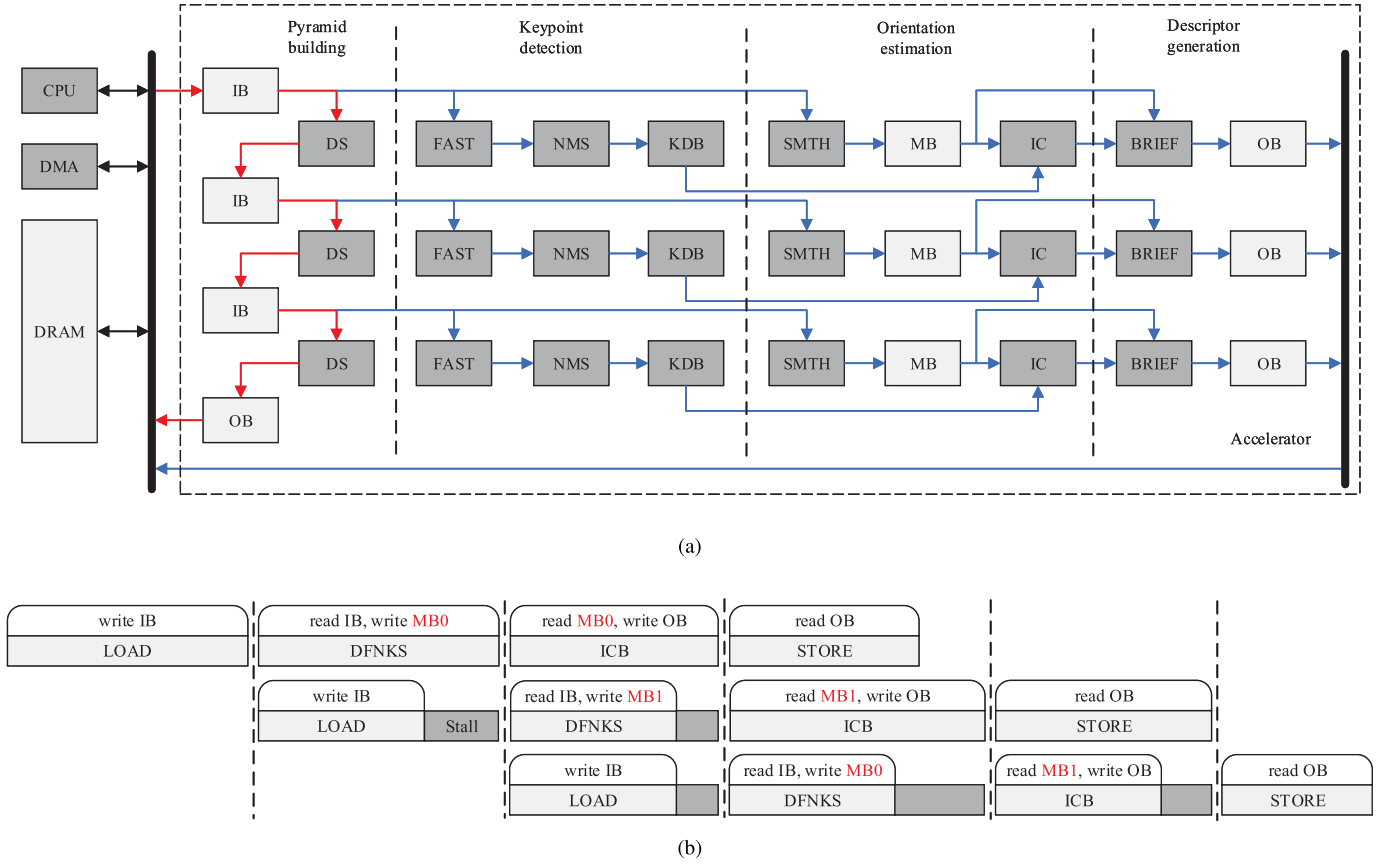


Fig. 3. Diagram of (a) image feature extraction system (IB: input buffer; descriptor: down-sampling; NMS: non-max suppression; KDB: keypoint distribution balance; SMTH: smoothing; MB: middle buffer; IC: intensity centroid; OB: output buffer) and (b) pipeline timing (DFNKS: down-sampling, FAST, non-max suppression, keypoint distribution balance and smoothing; ICB: intensity centroid and BRIEF).

A. Serial-Parallel Hybrid Workflow

The initial idea derives from the article in [17]. The authors proposed a time-sharing pyramid pipeline, where images are processed from the coarsest scale to the finest scale on the shared processing element (PE) in series scheduled by the timing MUX. The sequential processing of multiple scales on a shared functional unit will delay the processing. It is not a significant problem in [17] because its scaling factor is 2. The processing time of higher scales contributes only a small part of the total timing cost. For our hardware ORB processing, a scaling factor of 1.25 is used to approximate the default value 1.2 in the OpenCV library [16]. The processing time of higher scales is much larger than that of [17]. In this article, three scales as an octave are processed on three parallel pipelines. To process multiple octaves in serial, the DRAM plays the role of a timing MUX by storing the lowest scale of each octave. As shown in Fig. 3(a), the proposed architecture consists of three identical pipelines. Each pipeline consists of a streaming frontend including pyramid building and keypoint detection, and a non-streaming backend including orientation estimation and descriptor generation. The image pyramid is built in series (see the red arrows) since the higher scale is down-sampled from its previous lower scale. If more than one octave is needed, the output of the last down-sampling will be written to the DRAM as the lowest scale of the next octave for later

use. When the down-sampling on the current block completes, the rest operations are performed on the three scales in parallel (see the blue arrows), and the final results (descriptors) are written to the DRAM. After this, the lowest scale of the next octave is loaded from the DRAM and processed. Therefore, the architecture can support any number of scales by repeating the above steps. Except for the source image, the lowest scale of each octave can share the same DRAM space because they are processed in series. As a result, the requirement of DRAM capacity is limited even if the number of scales increases.

The timing diagram of a pipeline is shown in Fig. 3(b). At the frontend of the pipeline, the down-sampling unit, the FAST unit, the NMS unit, the KDB unit, and the SMTH unit can be grouped into one-stage DFNKS because they article on the same input block simultaneously. Similarly, the IC unit and the BRIEF unit at the backend are also looked as one stage ICB. Therefore, processing of a block has four steps, including LOAD, DFS, ICB, and STORE. Each step is started and stopped by the handshaking signals. Since the frontend is streaming and the backend is nonstreaming, the workload imbalance between DFS and ICB will cause the pipeline stall. Assume that at the most 32 keypoints are detected in a 75×75 area of each 125×125 block (see Section III-B), and computing the orientation and descriptor of a keypoint requires 220 cycles (see Sections IV-B and IV-C). It takes about 15 000 cycles and 7000 cycles for the

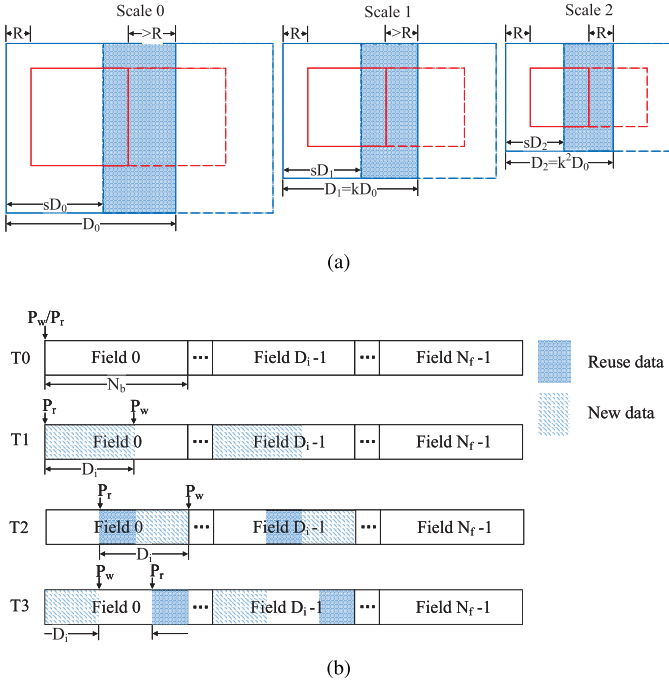


Fig. 4. Diagram of (a) block-wise reusable dataflow and (b) data placement in the reuse buffer.

frontend and the backend to process a block, respectively, indicating that the backend works only for half the time. Hence, we place the loaded block in a three-bank buffer and duplicate computing logics in FAST detection of each pipeline. Besides, the maximum number of keypoints in a block is limited to 24. For a full-HD image, about $1920/75 \times 1080/75 \times 24 = 8250$ features can be extracted at most, which is sufficient for most computer vision applications. On each scale, the FAST unit produces the detection results of three rows. After this, the NMS unit selects a keypoint candidate with the highest score in a 3×3 window every three cycles. Moreover, two banks are used to store the smoothed block as a ping-pong buffer. As a result, the overall throughput is much improved (see Section V-B).

B. Block-Wise Reusable Dataflow

Based on the series-parallel hybrid workflow, the block-wise reusable dataflow is shown in Fig. 4(a). Assume that B_j^i stands for the i th block on the j th scale, D_j is the width (height) of the block B_j^i and the inverse scaling ratio is k . In the beginning, a $D_0 \times D_0$ block B_0^0 is loaded to the on-chip buffer, indicated by the solid blue box. Then, it is downsampled by $1/k$ to produce a $D_1 \times D_1$ block B_1^0 , where $D_1 = kD_0$. Similarly, a $D_2 \times D_2$ block B_2^0 is downsampled on B_1^0 , where $D_2 = kD_1 = k^2D_0$. In Section II-D, we know that the BRIEF descriptor is computed in a circular area centered at the keypoint with a radius of R . Therefore, the pixels near the block border cannot be used for keypoint detection. Otherwise, no enough valid pixels in the on-chip buffer can be used for descriptor generation. Hence, the FAST detector has to operate in a smaller area of the block, denoted by the solid red box. The border distance of the block and its FAST

detection area (hereinafter referred to as the margin) should be greater than or equal to R , which is defined as the “overlap condition.” As the image is scanned from the top left to the bottom right, we fix the top and left margins to R . In order to detect the pixels near the border, the next block (see the red and blue dotted boxes) must be overlapped with the previous (see the blue shadow region). Once the overlap condition on the highest scale is satisfied, it will be easily satisfied for the lower scales by enlarging their overlap widths. To regulate the dataflow on three pipelines (Scale 0 to Scale 2), the same nonoverlapping ratio s is applied to the three scales. Thus, the overlap condition on Scale 2 is formulated as

$$k^2D_0 - sk^2D_0 = 2R \mid D_0, k^3D_0 \in N^* \quad (5)$$

which indicates that on Scale 2 the right margin is also R , that is, the width of the overlapped area equals exactly to $2R$, and the block width on each scale is a positive integer. If the inverse scaling ratio k and the margin R are set to 0.8 and 16, respectively, then we can figure out a set of reasonable value (0.6, 125) for (s, D_0) at moderate on-chip memory cost. That is to say, the size of blocks loaded from the DRAM is 125×125 , and keypoints are detected in a 75×75 area in such a block, and 40% pixels close to the right border in a block are reused in the on-chip buffer. When it returns to the leftmost, a new block has to be loaded.

We also design a multifield block buffer for each scale to support data reuse. As shown in Fig. 4(b), the buffer is divided into N_f fields and each field can store N_b data, where $N_f \geq D_j$ and $N_b \geq D_j$ ($j = 0, 1, 2$). On the j th scale, a write pointer P_w and a read pointer P_r are attached to each field of the buffer. When a $D_j \times D_j$ block is stored in the buffer, D_j rows are written to D_j fields in order. Inside a field, a row of data is written starting from P_w and then read out from P_r . After that, the two pointers are updated to the end current stored

$$P'_{w/r} = \begin{cases} P_{w/r} + U_j^i & P_{w/r} + U_j^i \leq N_b \\ P_{w/r} + U_j^i - N_b, & \text{else} \end{cases} \quad (6)$$

$$U_j^i = \begin{cases} D_j, & i = 0 \\ sD_j, & \text{else} \end{cases} \quad (7)$$

where U_j^i is the nonoverlapping width on the j th scale. Note that the update of P_r always falls behind P_w by a buffer-writing phase. The write and read addresses $A_{w/r}$ are given by

$$A_{w/r} = \begin{cases} P_{w/r} + C_y \times N_b + C_x & P_{w/r} + C_x \leq N_b \\ P_{w/r} + C_y \times N_b + C_x - N_b, & \text{else} \end{cases} \quad (8)$$

where C_x and C_y are, respectively, the horizontal and vertical coordinates of current pixel in a block. This procedure is repeated until all blocks in a “macro row” have been scanned and then P_r and P_w are set to 0 again.

IV. HARDWARE IMPLEMENTATION

To improve the robustness and hardware performance, we further propose three novel optimization ideas: 1) keypoint distribution balance (KDB); 2) approximate quantization; and

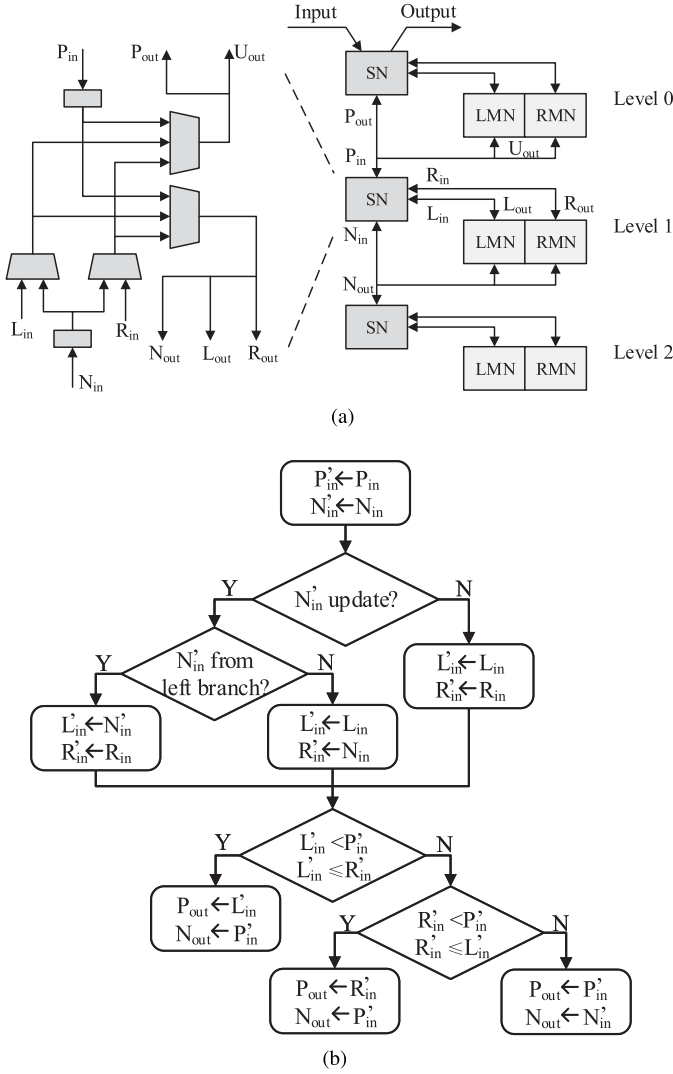


Fig. 5. Architecture of (a) heap sort (SN: switch node; LMN: left memory node; RMN: right memory node) and (b) operations performed by SN.

3) superscalar processing. In this section, we describe the implementation of these features in detail. The architecture of the SMTH unit and the FAST detection unit can be found in our previous works [14], [15], and the NMS unit is simply a set of CMPs similar to [17].

A. Two-Dimensional KDB

To avoid massive keypoints gathering in a block while few in others, it is necessary to balance the number of keypoints in different blocks over a frame. We adopt a heap sort unit (HSU) based on dual-port SRAM proposed in [18] to meet the requirement. As shown in Fig. 5(a), the HSU is a multilevel binary tree. Each level contains two dual-port SRAMs as left and right nodes, and a switch node (SN) to exchange data between the parent node and its leaf nodes. The capacity of memory nodes at level 1 is double that of level 0 and so on. Data stored in a memory node has two fields for the score and the coordinates of a keypoint, respectively. Before the HSU is used for descending sort, the memory nodes at each level are

filled by initial value with the “highest” score. When data swap occurs in an SN, it takes one clock cycle to update the parent node (U_{out}) and left (L_{out}) or right (R_{out}) node in the SRAM. For the SN at the next (previous) level, not until the parent (leaf) node SRAM has been written can it fetch the updated data for a swap. To reduce this latency, the updated data is not only sent to SRAM, but also the SN through a forward path (P_{in}, P_{out}, N_{in} and N_{out}).

Operations performed by each SN are shown in Fig. 5(b). When updated data P_{in} and N_{in} enter an SN, they are first registered as P'_{in} and N'_{in} . The left leaf L'_{in} and the right leaf R'_{in} are selected from SRAM or N'_{in} . Then the scores of P'_{in} , L'_{in} , and R'_{in} are compared and P'_{in} is swapped with either L'_{in} or R'_{in} or not. An input data is sorted after all the SNs complete swap and then stored at the correct location of the memory node. The sorted data can be read out in sequence by feeding a flush data with the “lowest” score to the tree. Since each SN costs two cycles for a swap, the unsorted data has to be input every other cycle at least. Fortunately, the timing of keypoints filtered by 3×3 NMS meets this requirement. After NMS, the horizontal and vertical coordinates of keypoints with their scores enter the HSU and are sorted in descending order. A threshold of N_{th} is set to the HSU and if the number of keypoints is more than N_{th} , then only first N_{th} keypoints with higher scores will be retained.

B. Approximate Computation in Orientation Estimation

Hardware implementation of orientation estimation has two difficulties: 1) large amounts of memory access and 2) complex operations such as division and inverse trigonometric function. We adopt approximate computation combined with multibank memory to address these issues.

As shown in Fig. 6(a), the IC is computed in a circular area centered at the keypoint with a radius of R ($R = 16$). In the circular area, every other pixel is sampled on both x and y directions. Thus, the memory access is reduced to about 25% of full sampling. To further reduce the processing time, the smoothed image block is interleaved in a four-bank memory. Since the sampled pixels are distributed symmetrically to the coordinate axis, we allocate the pixels to different banks based on the parity of the second LSB of their x - y coordinates (M_{ee}, M_{oe}, M_{oo} and M_{eo} in Fig. 6(b)). Therefore, every four consecutive sampled pixels (called a “cell”) are stored in different banks, and they can be simultaneously accessed without conflicts. Besides, the pixels on the axis are grouped into four neighbor quadrants. When computing the IC, the circular area is scanned from one quadrant to another. Inside a quadrant, the cells are loaded one by one [see the blue arrow and box in Fig. 6(a)]. As a result, the total timing cost is reduced to 6% after optimization.

The sampling pattern is represented by its innermost pixel. Specifically, the absolute value of its x - and y -coordinates are encoded as a combination of three two-bit fields, and each field stands for a power of two. For example, if the x -coordinate is -14 , and it is encoded as $\{11, 10, 01\}$ ($\{3, 2, 1\}$ in Dec). This coordinate is recovered by calculating $2^3 + 2^2 + 2^1$ and checking the quadrant, and the coordinates of the other three

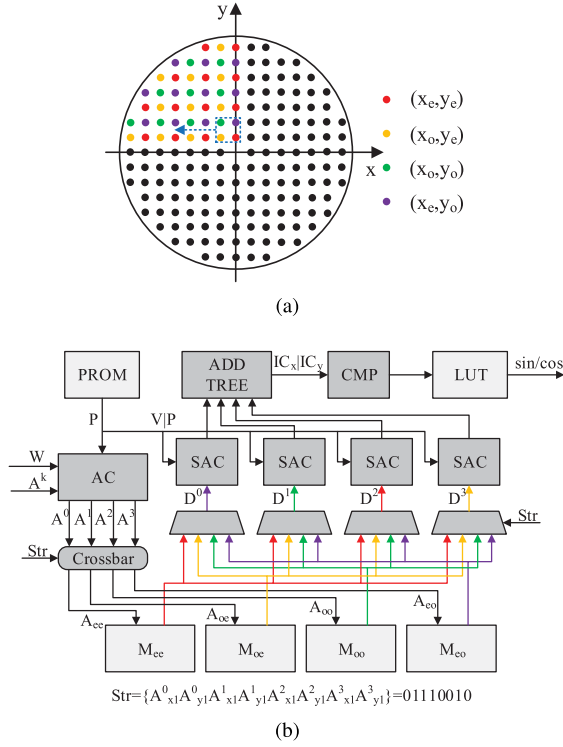


Fig. 6. (a) Quadrant-aware discrete sampling. (b) Architecture of the IC unit (AC: address conversion; SAC: shift and accumulation; PROM: pattern ROM).

pixels differ from the innermost by two. Near the circular border, maybe less than four pixels in a cell are needed for computation. Thus, we use the extra four bits to indicate where the four pixels are valid. The architecture of the IC unit is shown in Fig. 6(b). The sampling patterns (P) with valid flags (V) are prestored in PROM. When a keypoint arrives, the first sampling pattern with the valid flag is read from PROM. The location of the four pixels (A^0 to A^3) in the block coordinate system is calculated by an address conversion unit (AC) using the sampling pattern (P), the width of the image block (W) and the coordinates of the keypoint (A^k). The second LSB of each pixel's x - y coordinates form an eight-bit string (Str), and the four banks (M_{ee} to M_{eo}) are accessed under the control of the string. Then four pixels (D_0 to D_3) are fetched from the four-bank memory and used to compute the partial sums. Due to the encoding method above, the partial sum of the IC's x - and y -components (IC_x , IC_y) can be easily calculated by the shift accumulation units (SAC) and an addition tree.

When the IC computation completes, the orientation (angle) is estimated using approximate quantification, and its sine and cosine are obtained by indexing a lookup table (LUT). The angle of 0° to 90° in a quadrant is quantified to nine intervals with an increment of 11.25° (see the first two columns in Table I). The tangent of each interval is multiplied by 32 and fixed point as Q_t . The IC's y -component is also multiplied by 32 as IC_{y32} . The x -component is scaled up to $\times 3$ (IC_{x3}), $\times 10$ (IC_{x10}), and so on. IC_{y32} is compared with IC_{x3} to IC_{x324} to estimate the angle. For example, if $IC_{y32} \geq IC_{x3}$ and $IC_{y32} < IC_{x10}$, then the tangent value

TABLE I
LOOKUP TABLE FOR ORIENTATION

Angle	Sector	Q_t	Index	Q_s	Q_c
0	[0, 5.36)	[0, 3)	0	0	1024
11.25	[5.36, 17.35)	[3, 10)	1	200	1004
22.5	[17.35, 27.98)	[10, 17)	2	392	946
33.75	[27.98, 39.09)	[17, 26)	3	569	851
45	[39.09, 51.34)	[26, 40)	4	724	724
56.25	[51.34, 61.93)	[40, 60)	5	851	569
67.5	[61.93, 72.90)	[60, 104)	6	946	392
78.75	[72.90, 84.36)	[104, 324)	7	1004	200
90	[84.36, 90]	[324, $+\infty$)	8	1024	0

TABLE II
HARDWARE AND TIME COST IN ORIENTATION ESTIMATION

Items	Non-opt	Proposed
ADD/SUB	706	47
MUL	706	0
DIV	1	0
Tri Fun	3	0
Shift	0	32
CMP	0	16
LUT(bit)	0	180
Time(cycle)	720	52

falls into the second interval of Q_t . The sine and cosine values are obtained from the second entry of the LUT. The LUT has nine 20-bit entries, and each of them stores a pair of sine and cosine enlarged by 1024 times. The sine and cosine are used to rotate the coordinates of the sampled pixels before computing the BRIEF descriptor. Therefore, complex operations such as multiplication, division, and triangle functions are replaced by addition/subtraction, shift, comparison, and LUT, and the processing time is significantly reduced (see Table II).

C. Superscalar Processing in Descriptor Generation

Descriptor generation is another bottleneck because hundreds of pixels with random coordinates are read from memory for one keypoint. To reduce the processing time, we make use of the same four-bank memory and design a four-issue out-of-order (OOO) superscalar architecture for descriptor generation. The superscalar pipeline is composed of six stages, as shown in Fig. 7 (some pipeline registers and control signals are omitted for brevity).

- 1) *Instruction Fetch (IF)*: Computation of a 256-bit descriptor needs select 512 pixels from the four-bank memory, where four pixels can be accessed per cycle at most. Therefore, every four pixels' coordinates (patterns) are formulated as an instruction. The 128 instructions are fetched from the instruction memory (IM) under the control of the program counter (PC).
- 2) *Instruction Decode (ID)*: Four patterns are rotated by the angle of the keypoint and converted into global patterns in the block coordinate system. The global patterns are allocated to four four-entry reservation stations (RS)

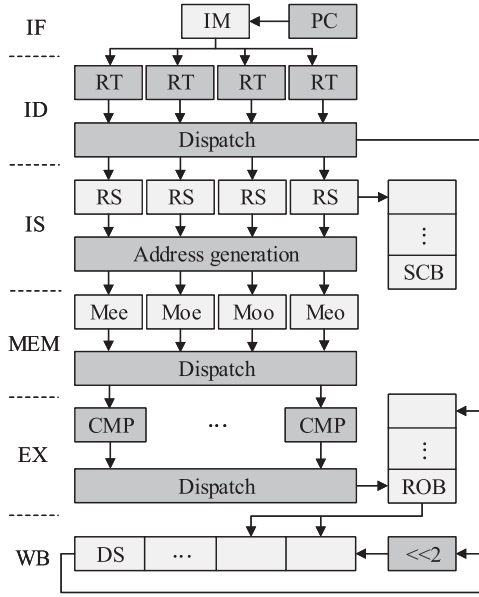


Fig. 7. Superscalar pipeline for descriptor generation (PC: program counter; IM: instruction memory; RT: rotation; RS: reservation station; SCB: scoreboard; ROB: reorder buffer; CMP: CMPs; DS: descriptor).

based on the parity of patterns' x - y coordinates. If any RS has no enough space for new patterns, reading IM and writing RS are stalled. Besides, new patterns are also written to the reorder buffer (ROB) in turn to guarantee an in-order commit.

- 3) *Issue (IS)*: The result of decoding an instruction is cached in RS, and there are four fields for each entry of an RS:
 - a) Bit—position of descriptor bit to be computed.
 - b) Port—input port of the CMP to be used by the pixel.
 - c) X— x -component of the pixel's global coordinate.
 - d) Y— y -component of the pixel's global coordinate.

The memory address of a pixel is computed by the address generation unit with RS.X and RS.Y. Then it is issued to the four-bank memory, which is also shared by the orientation estimation unit, to access the sampled pixel.

- 4) *Memory Access (MEM)*: The status of instructions and CMPs is tracked by an eight-entry scoreboard (SCB). Each entry with four fields is associated with a CMP:
 - a) Busy—indicates whether the CMP is used.
 - b) Bit—position of descriptor bit to be computed.
 - c) R0,R1—indicate whether two inputs of the CMP are ready.

The pixel read from the memory is dispatched to one of the two inputs specified by RS.Port of an available CMPs. In the meanwhile, SCB.Busy and SCB.R0 or SCB.R1 are set to YES, and RS.Bit is copied to SCB.Bit. These parameters together help designate a CMP correctly.

- 5) *Execute (EX)*: The simulation results show that no more than seven CMPs are active at one cycle in the four-issue OOO architecture. Thus, we implement eight CMPs

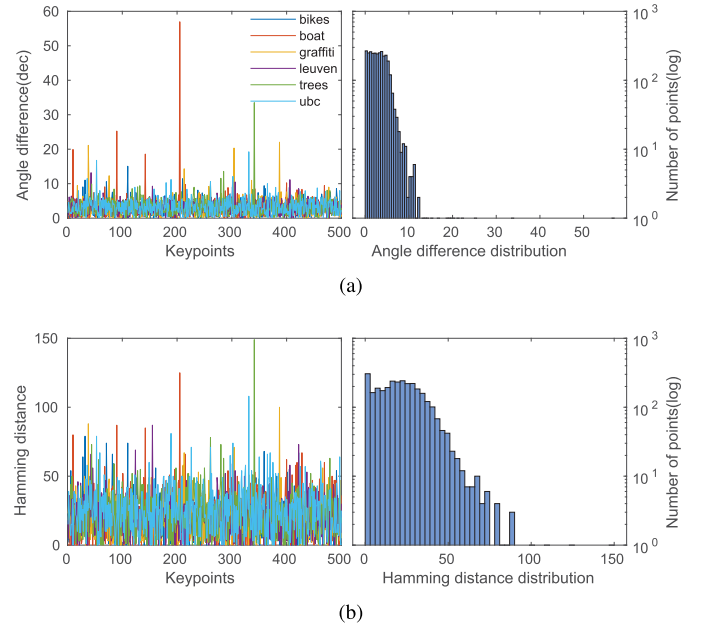


Fig. 8. Approximation error in the orientations and its impact on descriptors. (a) Absolute difference of angles. (b) Hamming distance of descriptors.

to avoid the structural hazard. As a result, the CMPs perform OOO execution for the descriptor bits, which have to be rearranged by the ROB.

- 6) *Write Back (WB)*: The 16-entry ROB is used to in-order commit the results and each entry includes four fields:
 - a) Busy—indicates whether the entry is used.
 - b) V—indicates whether the result is valid.
 - c) Bit—position of descriptor bit to be written.
 - d) R—comparison result or descriptor bit.

At the ID stage, two ROB entries have been selected to record the instruction, that is, ROB.Busy and ROB.Bit of two entries have been set in sequence. Then the comparison result is written into ROB.R of the entry with the same value of ROB.Bit as SCB.Bit, and ROB.V is set to YES as well. Finally, the descriptor bits are read from ROB two by two and written back to the register in order.

Simulation shows that the superscalar architecture completely generates one 256-bit descriptor in 167 cycles on average, which reduces the processing time by 67%.

V. EXPERIMENTAL RESULTS

In this section, we evaluate memory and bandwidth of the proposed architecture, impact of the optimizations on the accuracy, and the performance compared with previous FAST-BRIEF accelerators. All the accuracy evaluations are performed on a function-accurate C++ simulator of the proposed accelerator and the Oxford VGG data set [19].

A. Feature Quality and Robustness

The proposed hardware ORB processing has approximation only in the orientation estimation, which causes the errors of descriptors. The other steps including pyramid building, key-point detection, and descriptor generation exactly follow the

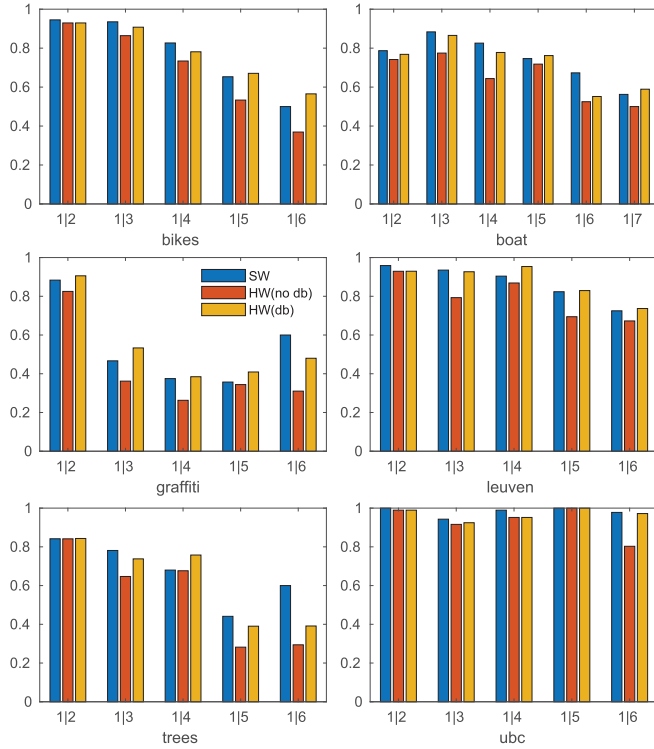


Fig. 9. Valid match ratio test on the six image sets (x -axis stands for two matched images and y -axis is valid match ratio).

initial ORB algorithm [5]. Therefore, to evaluate the approximation errors, we measure the difference in the keypoint orientations (angles) and the Hamming distance of the descriptors computed by the hardware (simulator) and the software (OpenCV). We select one image from each category (bikes, boat, graffiti, Louvain, trees, and ubc) of the VGG data set, and 500 FAST keypoints with a threshold of 20 are extracted from each image by the hardware. Then the same keypoints are used to compute the IC angles and BRIEF descriptors by the hardware and software, respectively. As shown in Fig. 8(a), most of the angles computed by the hardware and software are very close, and the average difference is about 3.19° . Besides, the Hamming distance of two descriptors computed on the same keypoint by the hardware and software is shown in Fig. 8(b). The larger the difference in the keypoint angles, the larger the Hamming distance between the descriptors is. It indicates that the approximation in the orientation estimation has a significant impact on the descriptor quality. The other hardware specifications, such as the number of quantified angles, the bit-width for centroid moments, and sin/cos values, are already evaluated in [10], and we directly select the optimal values.

Furthermore, we learned from [7] and [20] to measure the robustness of features using the valid match ratio, that is, the number of correctly matched features to the number of all matched features. As shown in Fig. 9, we calculate the valid match ratio of the ORB features extracted by the software, hardware without distribution balance, and hardware with distribution balance, respectively. In each of the six image sets, one image is taken as the reference to be matched with the others. Due to approximate computing in the orientation

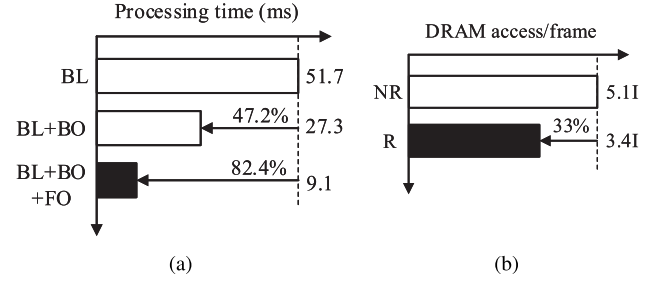


Fig. 10. Ratio of (a) throughput increase and (b) DRAM access reduction (BL: baseline; BO: backend optimization; FO: frontend optimization; I: the number of pixels in a frame).

TABLE III
FPGA RESOURCE UTILIZATION AND DYNAMIC POWER BREAKDOWN

Item	LUTs	REGs	DSPs	BRAMs(18Kb)	Power(mW)
DS	354	321	0	1.5	9
FAST	319	354	0	0	5
NMS	84	67	0	0	1
KDB	301	287	0	0	4
SMTH	1284	229	0	0	43
IC	1085	245	4	1	53
BRIEF	2649	511	7	1	79

TABLE IV
COMPARISON OF FPGA RESOURCE UTILIZATION

Work	Device	LUTs/LEs	REGs	DSPs	MEM(Mb)
[9]	XC5VLX3	12368	10156	0	1.76
[12]	Arria V	206000	231973	449	8.18
[11]	Stratix V	25648	21791	8	9.44
[10]*	5AGXFB3	12523	10019	0	0.11
this work	XCZU9EG	28168	9528	33	1.47
(%)		(15.75)	(2.32)	(1.90)	(9.16)

*Only the rBRIEF unit is implemented, and row buffers are not calculated

TABLE V
PROCESSING SPEED OF SOFTWARE AND HARDWARE FOR VGA IMAGES

Image	SW(ms)	HW(ms)	SW _{FHD} (ms)	HW _{FHD} (ms)
bikes	62	3.10	197	9.32
boat	93	3.05	218	9.25
graffiti	94	3.31	224	9.23
leuven	47	3.26	182	9.29
trees	69	3.21	204	9.22
ubc	78	2.98	213	9.21
Average(fps)	13	317	5	108

estimation, the valid match ratio of the hardware without KDB decreases slightly compared to that of the software for most cases. When KDB is adopted, the valid match ratio of the hardware is increased by 7.09% on average, and in some image sets, it is even better than that of the software. Intuitively, if there is no distribution balance to keypoint detection, the final good keypoints trend to gather in one or more small regions. Due to the similar pattern information carried by these keypoints, the descriptors are likely to be less distinctive.

TABLE VI
PERFORMANCE COMPARISON WITH RELATED WORKS

Work	Algorithm	Architecture	Implementation	Feature	Performance	Power(mW)
[7]	FAST-BRIEF	non-streaming, row-wise	ASIC, 130nm, 78.3k gates, 128kB SRAM	0 scales, 0 angles, no NMS, no KDB	122fps, FHD, 200MHz	182
[9]	ORB-like	non-streaming, row-wise	FPGA, XC5VLX3, 0 DSPs, 12368 LUTs, 10156 REGs, 205kB BRAM	3 scales, 16 angles, 4×4 NMS, no KDB	135fps, FHD, 200MHz	-
[12]	ORB	streaming, row-wise	FPGA, Arria V GX, 449 DSPs, 206000 LEs, 231973 REGs, 1047kB BRAM	9 scales, 64 angles, no NMS, no KDB	110.9fps, FHD, 230MHz	5340
[11]	ORB	streaming, row-wise	FPGA, Stratix V, 8 DSPs, 25648 LUTs, 21791 REGs, 1208kB BRAM	2 scales, no NMS, no KDB	67fps, VGA, 203MHz	4559
[8]	FAST-BRIEF	streaming, row-wise	ASIC, 65nm, 28kB SRAM	0 scales, 0 angles, 7×7 NMS, no KDB	2170fps, VGA	1131
This work	ORB	non-streaming, block-wise	FPGA, XCZU9EG, 33 DSPs, 28168 LUTs, 9528 REGs, 188kB BRAM	3 scales, 32 angles, 3×3 NMS, KDB	108fps, FHD, 200MHz	873

On the contrary, the farther they are away from each other, the lower the similarity and the less mismatch. Therefore, the 2-D KDB helps improve the matching robustness.

B. Throughput and DRAM Access

As shown in Fig. 10(a), we evaluate the throughput improvement by the parallel processing in the FAST, IC, and BRIEF units. The timing cost is estimated on the first pipeline since the block size is the largest. Assume that the block size is 125×125 , the number of keypoints in each block is 24 at most, the number of blocks to fill a row and a column in a full-HD image are about 25 and 14, respectively, and the clock frequency is 200 MHz. Consider a baseline architecture (BL) without any parallel processing on each pipeline. The frontend can process a block in $125 \times 125 \times 5 \text{ ns} = 78.1 \mu\text{s}$. At the backend, computing the orientation and the descriptor of 24 keypoints requires $24 \times (720 + 512) \times 5 \text{ ns} = 147.8 \mu\text{s}$. Thus, it takes about $147.8 \times 25 \times 14 \mu\text{s} = 51.7 \text{ ms}$ to process a full-HD image. With the optimizations in the IC and BRIEF units (BL+BO), the backend can process a keypoint in 220 cycles on average. It completes a block at most in $220 \times 24 \times 5 \text{ ns} = 26.4 \mu\text{s}$, less than the $78.1 \mu\text{s}$ at the frontend. Thus, the total processing time is dominated by the frontend as $78.1 \times 25 \times 14 \mu\text{s} = 27.3 \text{ ms}$. Furthermore, with the FAST unit detecting three pixels in parallel (BL+BO+FO), the processing time of a block at the frontend is reduced to $78.1/3 \mu\text{s} = 26 \mu\text{s}$, comparable to that at the backend. As a result, the total timing cost is about $27.3/3 \text{ ms} = 9.1 \mu\text{s}$, reduced by 82.4% compared to the baseline architecture and achieving a speed of 108 fps.

Besides, the ratio of the DRAM bandwidth reduction by the block-wise reusable dataflow is also given in Fig. 10(b). Assume that the block width is D , the image size is $I = W \times H$, the nonoverlapped ratio s is 0.6, and the inverse scaling ratio k is 0.8. Then the size of the overlapped part of two adjacent blocks is $(1 - s)D^2$, and the number of blocks to fill a row and a column are $W/(sD)$ and $H/(sD)$, respectively. In our design, data is only reused in the horizontal direction. Thus, the number of reloaded pixels in an image is $WH(1 - s)D^2/(sD)^2 = 1.25I$. If no data is reused at all, it will be doubled to $2.5I$. To process a pyramid with many

scales S_0 to S_n , S_0 is read from the DRAM, and S_3, S_6, S_9 , and so on are written to and read from the DRAM. Note that when down-sampled images are written to the DRAM, only the nonoverlapped part of a block is stored. Therefore, the amount of DRAM access is $2.25I + 3.25k^6I + 3.25k^{12}I + 3.25k^{18}I \dots = 3.4I$ in total. If no data is reused, then it will be $5.1I$. Thus, the proposed dataflow can save 33% DRAM bandwidth compared with nondata reuse. When the accelerator processes full-HD images at 108 fps, it requires 6.1-Gb/s DRAM bandwidth at most.

C. Hardware Cost and Performance

The proposed accelerator is developed using Vivado and implemented on the FPGA side (XCZU9EG) of a Xilinx Zynq Ultrascale+ MPSoC (ZCU102), which is very suitable for the system architecture shown in Fig. 3(a). The synthesis options are set by default, and a SAIF file generated by postimplementation function simulation is used to perform power estimation. Table III reports the resource utilization and dynamic power dissipation of each function unit at an operating frequency of 200 MHz. The BRIEF unit consumes the most LUTs, Registers, and DSPs as well as the dynamic power due to the complexity in superscalar architecture. The LUTs and Registers are used for the data structures such as SCBs, RSs, and ROBs, and DSPs are used in address generation to access these data structures and the 4-bank image buffer. The overall hardware cost of the accelerator is compared with related FPGA-based works in Table IV. Although more LUTs and DSPs are used in the IC and BRIEF units to enable faster processing, the accelerator still cost very few FPGA resources.

To compare the performance with the software solution, we program the ORB algorithm using OpenCV [16] on an Intel i5-4670 processor at 3.40 GHz with 8-GB DRAM. The scaling factor and the patch size are set to 1.25 and 32, respectively, and about 1000 features are extracted on eight scales per image. Table V gives the processing time on the VGG image sets, which are also resized to full-HD resolution. The accelerator achieves 124 and 35 fps under VGA and full-HD resolutions, showing much higher efficiency than that of software. Besides, the performance of this article is

also compared to that of the related hardware accelerators in Table VI. This article in [8] achieves exceptionally high speed by fine-grained optimization at circuit level, but it has no scale invariance and rotation invariance. This issue also occurs in [7] because both of them adopt the FAST-BRIEF algorithm. Compared to the work in [11], this article performs better in the aspect of resolution, speed, and power. The work in [12] has almost the same speed to ours, but it has no keypoint NMS and costs more power even if normalized to the same specification of three scales. Although the processing speed and resource overhead in [9] outperforms ours, this article still has advantages over it. In this article, the ORB algorithm is retained to the largest and the 2-D KDB is implemented, which improves the feature matching robustness. Duo to the combination of series-parallel workflow and block-wise dataflow, this article is flexible to process any number of scales and arbitrary resolution. Compared to [9], this article still achieves 108 fps for full-HD images at a moderate hardware cost.

VI. CONCLUSION

This article presented a flexible and efficient hardware accelerator for ORB-based image feature extraction. Using external memory as the frame buffer, the nonstreaming architecture can process images with any resolutions and scales. In combination with the series-parallel hybrid pipelines, the block-wise reusable dataflow saves about 33% external memory bandwidth and realizes 2-D KDB, which improves the valid feature match ratio by 7.09%. The above architecture and dataflow can also be used for the other image feature extraction algorithms like SIFT and SURF. Besides, the parallel processing at the frontend of each pipeline increases $2\times$ throughput. In addition, the approximate computing and superscalar processing reduce the timing cost in orientation estimation and descriptor generation by 92.78 and 67%, respectively. As a result, the overall processing time is reduced by 82.4% compared to that of the nonoptimized baseline architecture. The accelerator is implemented in an FPGA SoC, and it achieves 108 fps for full-HD images at 200 MHz under 873 mW.

REFERENCES

- [1] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *Int. J. Comput. Vis.*, vol. 60, no. 2, pp. 91–110, 2004.
- [2] H. Bay, T. Tuytelaars, and L. Van Gool, "SURF: Speeded up robust features," in *Computer Vision-ECCV 2006*. Berlin, Germany: Springer, 2006, pp. 404–417.
- [3] E. Rosten, R. Porter, and T. Drummond, "Faster and better: A machine learning approach to corner detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 32, no. 1, pp. 105–119, Jan. 2010, doi: [10.1109/TPAMI.2008.275](https://doi.org/10.1109/TPAMI.2008.275).
- [4] M. Calonder, V. Lepetit, C. Strecha, and P. Fua, "BRIEF: Binary robust independent elementary features," in *Computer Vision-ECCV 2010*, 2010, pp. 778–792.
- [5] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, "ORB: An efficient alternative to SIFT or SURF," in *Proc. IEEE Int. Conf. Comput.*, Nov. 2011, pp. 2564–2571.
- [6] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM: A versatile and accurate monocular SLAM system," *IEEE Trans. Robot.*, vol. 31, no. 5, pp. 1147–1163, Oct. 2015.
- [7] J. Park, H. Kim, and L. Kim, "A 182 mw 94.3 f/s in full hd pattern-matching based image recognition accelerator for an embedded vision system in 0.13- μ m CMOS technology," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 23, no. 5, pp. 832–845, May 2013.
- [8] S.-K. Lam, G. Jiang, M. Wu, and B. Cao, "Area-time efficient streaming architecture for FAST and BRIEF detector," *IEEE Trans. Circuits Syst., II, Exp. Briefs*, vol. 66, no. 2, pp. 282–286, Feb. 2019.
- [9] W. Zhu, L. Liu, G. Jiang, S. Yin, and S. Wei, "A 135-frames/s 1080p 87.5-mw binary-descriptor-based image feature extraction accelerator," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 26, no. 8, pp. 1532–1543, Aug. 2016.
- [10] T. H. Pham, P. Tran, and S.-K. Lam, "High-throughput and area-optimized architecture for rBRIEF feature extraction," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 4, pp. 747–756, Apr. 2019.
- [11] W. Fang, Y. Zhang, B. Yu, and S. Liu, "FPGA-based ORB feature extraction for real-time visual SLAM," in *Proc. Int. Conf. Field Program. Technol. (ICFPT)*, Dec. 2017, pp. 275–278.
- [12] J. Weberuss, L. Kleeman, D. Bolland, and T. Drummond, "FPGA acceleration of multilevel ORB feature extraction for computer vision," in *Proc. 27th Int. Conf. Field Program. Log. Appl. (FPL)*, Sep. 2017, pp. 1–8.
- [13] O. Ulusel, C. Picardo, C. B. Harris, S. Reda, and R. I. Bahar, "Hardware acceleration of feature detection and description algorithms on low-power embedded platforms," in *Proc. 26th Int. Conf. Field Program. Log. Appl. (FPL)*, Aug./Sep. 2016, pp. 1–9.
- [14] R. Sun, P. Liu, J. Wang, and Z. Zhou, "A low latency feature extraction accelerator with reduced internal memory," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2017, pp. 1–4.
- [15] R. Sun, P. Liu, J. Wang, C. Accetti, and A. A. Naqvi, "A 42 fps full-HD ORB feature extraction accelerator with reduced memory overhead," in *Proc. Int. Conf. Field Program. Technol. (ICFPT)*, Dec. 2017, pp. 183–190.
- [16] G. Bradski and A. Kaehler, *Learning OpenCV: Computer Vision in C++ With the OpenCV Library*, 2nd ed. Newton, MA, USA: O'Reilly Media, 2013.
- [17] Q. Zhu, N. Garg, Y.-T. Tsai, and K. Pulli, "An energy efficient time-sharing pyramid pipeline for multi-resolution computer vision," in *Proc. IFIP/IEEE 21st Int. Conf. Very Large Scale Integr. (VLSI-SoC)*, Oct. 2013, pp. 278–281.
- [18] W. M. Zabołotny, "Dual port memory based Heapsort implementation for FPGA," *Proc. SPIE*, vol. 8008, Oct. 2011, Art. no. 80080E.
- [19] K. Mikołajczyk and C. Schmid, "A performance evaluation of local descriptors," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, vol. 2, Jun. 2003, p. II.
- [20] D. Jeon *et al.*, "An energy efficient full-frame feature extraction accelerator with shift-latch FIFO in 28 nm CMOS," *IEEE J. Solid-State Circuits*, vol. 49, no. 5, pp. 1271–1284, May 2014.